

Ultimate guide to handle Big Datasets for Machine Learning using Dask (in Python)



Aishwarya Singh

Last Updated : 21 Oct, 2024



Introduction

Have you ever tried working with a large dataset on a 4GB RAM machine? It starts heating up while doing the simplest of machine learning tasks? This is a common problem data scientists face when working with restricted computational resources. One solution to optimize your workflow is to leverage tools like Dask Python, which efficiently handles large datasets by parallelizing operations and minimizing memory usage. This can significantly improve the performance of your machine learning tasks on resource-constrained systems.

When I embarked on my data science journey using Python, I quickly recognized that the existing libraries, such as Pandas and Numpy, have certain limitations when efficiently handling large datasets. While these libraries are undeniably powerful, their computational efficiency can be challenged, particularly when manipulating gigabytes of data. So, what steps can you take to overcome this obstacle? Enter Dask Python – a tool that seamlessly scales your data workflows, providing a flexible and parallel computing framework to tackle the challenges posed by extensive dataset. With Dask Python, you can harness the strength of parallel computing to process and analyze large volumes of data efficiently, making it a valuable addition to your data science toolkit.



This is where **Dask** weaves its magic! It works with Pandas dataframes and Numpy data structures to help you perform data wrangling and model building using large datasets on not-so-powerful machines. Once you start using Dask, you won't look back.

New Feature



Get Personalized Learning Path!

Set your goal and timeline. Get a path—under 2 mins.

[Create My Path](#)

In this article, we will look at what Dask is, how it works, and how you can use it for working on large datasets. We will also take up a dataset and put Dask to good use. Let's begin

Table of contents

1. Introduction
2. 1. A Simple Example to Understand Dask Python
3. 2. Challenges with Common Data Science Python Libraries (Numpy, Pandas, Sklearn)
4. 3. Introduction to Dask
5. 4. Set up your system: Dask Installation
 - 4.1 Using conda
 - 4.2 Using pip

1. A Simple Example to Understand Dask Python

Let me illustrate these aforementioned limitations with a simple example. Suppose you have 4 balls (of different colors) and you are asked to separate them within an hour (based on the color) into different buckets.



What if you are given a hundred balls and you have to separate them in an hour's time? That would be a tedious task but still sounds feasible. Imagine you are given a thousand balls and an hour to separate them into buckets. It is impossible for an individual to complete the task within the given time (in this case, the data is huge and the resources are limited). How would you accomplish this?

The best bet would be to ask a few other people for help. You can call 9 other friends, give each of them 100 balls and ask them to separate these based on the color. In this case, 10 people are simultaneously working on the assigned task and together would be able to complete it faster than a single person would have (here you had a huge amount of data which you distributed among a bunch of people).

Currently we use common libraries like pandas, numpy and scikit-learn for data preprocessing and model building. These libraries are not scalable and work on a single CPU. [Dask Python](#) however can scale up to a cluster of machines. To sum up,

pandas and numpy are like the individual trying to sort the balls alone, while the group of people working together represent **Dask**.

2. Challenges with Common Data Science Python Libraries (Numpy, Pandas, Sklearn)

Python is one of the most popular programming languages today and is widely used by data scientists and analysts across the globe. There are common python libraries (numpy, pandas, sklearn) for performing data science tasks and these are easy to understand and implement.

But when it comes to working with large datasets using these [python](#) libraries, the run time can become very high due to memory constraints. These libraries usually work well if the dataset fits into the existing RAM. But if we are given a large dataset to analyze (like 8/16/32 GB or beyond), it would be difficult to process and model it. Unfortunately, these popular libraries were not designed to scale beyond a single machine. It is like asking a single person to separate a thousand balls in a limited time frame, it's quite unfair to ask!

What should one do when faced with a dataset larger than what a single machine can process? This is where Dask comes into the picture. It is a python library that can handle moderately large datasets on a single CPU by using multiple cores of machines or on a cluster of machines (distributed computing).

3. Introduction to Dask

If you are familiar with pandas and numpy, you will find working with Dask fairly easy. [Dask](#) is popularly known as a 'parallel computing' python library that has been designed to run across multiple systems. Your next question would understandably be – what is parallel computing?

As in our example of separating the balls, 10 people doing the job simultaneously can be considered analogous to parallel computation. In technical terms, parallel computation is performing multiple tasks (or computations) simultaneously, using more than one resource.



Dask can efficiently perform parallel computations on a single machine using multi-core CPUs. For example, **if you have a quad core processor, Dask can effectively use all 4 cores of your system simultaneously for processing**. In order to use lesser memory during computations, Dask stores the complete data on the disk, and uses chunks of data (smaller parts, rather than the whole data) from the disk for processing. During the processing, the intermediate values generated (if any) are discarded as soon as possible, to save the memory consumption.

In summary, **Dask can run on a cluster of machines to process data efficiently** as it uses all the cores of the connected machines. One interesting fact here is that it is not necessary that all machines should have the same number of cores. If one system has 2 cores while the other has 4 cores, Dask can handle these variations internally.

Dask supports the Pandas dataframe and Numpy array data structures to analyze large datasets. Basically, Dask lets you scale pandas and numpy with minimum changes in your code format. How great is that?

4. Set up your system: Dask Installation

Before we go ahead and explore the various functionalities provided by Dask, we need to setup our system first. Dask can be [installed](#) with conda, with pip, or directly from the source. This section explores all three options.

4.1 Using conda

Dask is installed in Anaconda by default. You can update it using the following command:

```
conda install dask
```

4.2 Using pip

To install Dask using pip, simply use the below code in your command prompt/terminal window:

```
pip install "dask[complete]"
```

4.3 From source

To install Dask from source, follow these steps:

1. Clone the git repository

```
git clone https://github.com/dask/dask.git
cd dask
python setup.py install
```

2. Use pip to install all dependencies

```
pip install -e ".[complete]"
```

5. Dask Interface

Now that we are familiar with Dask and have set up our system, let us talk about the Dask interface before we jump over to the python code. [Dask](#) provides several user interfaces, each having a different set of parallel algorithms for distributed computing. For data science practitioners looking for scaling numpy, pandas and scikit-learn, following are the important user interfaces:

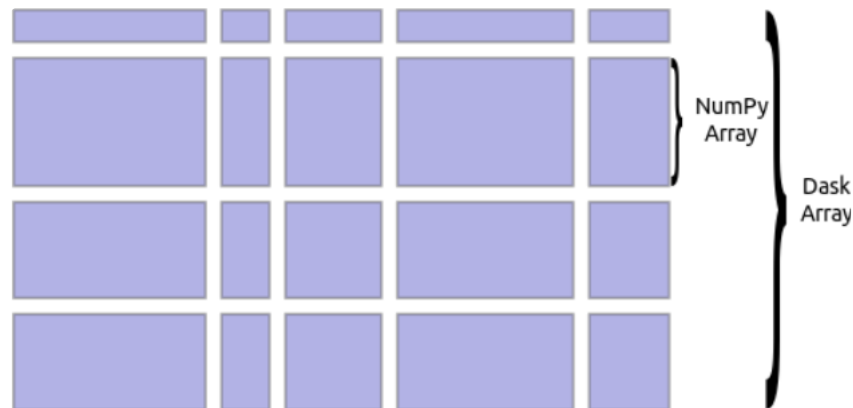
- **Arrays:** parallel Numpy
- **Dataframes:** parallel Pandas
- **Machine Learning:** parallel Scikit-Learn

The dataset used for implementation in this article is AV's [Black Friday](#) practice problem . You can download the dataset from the given link and follow along with the code blocks below. Let's get started!

5.1 Dask Arrays

A large numpy array is divided into smaller arrays which, when grouped together, form the Dask array. In simple words, Dask arrays are distributed numpy arrays! Every operation on a [Dask array](#) triggers operations on the smaller numpy arrays, each using a core on the machine. Thus all available cores are used simultaneously enabling computations on arrays which are larger than the memory size.

Below is an image to help you understand what a Dask array looks like:



As you can see, a number of numpy arrays are arranged into grids to form a Dask array. While creating a Dask array, you can specify the **chunk size which defines the size of the numpy arrays**. For instance, if you have 10 values in an array and you give the chunk size as 5, it will return 2 numpy arrays with 5 values each.

In summary, below are a few important features of Dask arrays below:

1. **Parallel:** Dask arrays use all the cores of the system
2. **Larger-than-memory:** Enables working on datasets that are larger than the memory available on the system (happens too often for me!). This is done by breaking the array into many small arrays and then performing the required operation
3. **Blocked Algorithms:** Perform large computations by performing many smaller computations. This is equivalent to sorting 1000 balls (large computation) by dividing it into 10 sets and sorting 100 balls (smaller computation)

We will now have a look at some simple cases for creating arrays using Dask.

1. Create a random array using Dask array

Python Code:

```
import dask.array as da

#using arange to create an array with values from 0 to 10
X = da.arange(11, chunks=5)
print(X.compute())

#to see size of each chunk
print(X.chunks)
```

As you can see here, I had 11 values in the array and I used the chunk size as 5. This distributed my array into three chunks, where the first and second blocks have 5 values each and the third one has 1 value.

2. Convert a numpy array to Dask array

```
import numpy as np
import dask.array as da
x = np.arange(10)
y = da.from_array(x, chunks=5)
y.compute() #results in a dask array

array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9])
```

Dask arrays support most of the numpy functions. For instance, you can use `.sum()` or `.mean()`, as we will do now.

3. Calculating mean of the first 100 numbers

```
import numpy as np
import dask.array as da

x = np.arange(1000) #arange is used to create array on values from 0 to 1000
y = da.from_array(x, chunks=(100)) #converting numpy array to dask array

y.mean().compute() #computing mean of the array

499.5
```

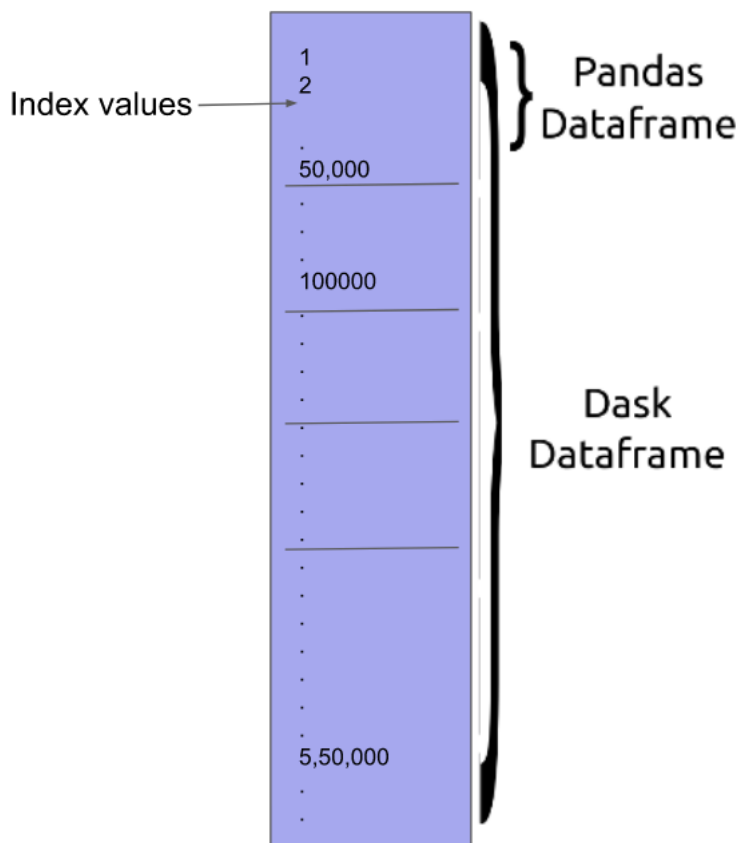
Here, we simply converted our numpy array into a Dask array and used `.mean()` to do the operation.

In all the above codes, you must have noticed that we used `.compute()` to get the results. This is because when we simply use `dask_array.mean()`, Dask builds a graph of tasks to be executed. To get the final result, we use the `.compute()` function which triggers the actual computations.

5.2 Dask Dataframe

We saw that multiple numpy arrays are grouped together to form a Dask array. Similar to a Dask array, a Dask dataframe consists of multiple smaller pandas dataframes. A large pandas dataframe splits row-wise to form multiple smaller dataframes. These smaller dataframes are present on a disk of a single machine, or multiple machines (thus allowing to store datasets of size larger than the memory). Each computation on a [Dask](#) dataframe parallelizes operations on the existing pandas dataframes.

Below is an image that represents the structure of a Dask dataframe:



The APIs offered by the Dask dataframe are very similar to that of the pandas dataframe.

Now, let's perform some basic operations on Dask dataframes. Time to load up the Black Friday dataset you had downloaded earlier!

1. Reading a csv file (comparing the read time with pandas)

```
#reading the file using pandas
import pandas as pd
%time temp = pd.read_csv("balckfriday_train.csv")

CPU times: user 485 ms, sys: 55.9 ms, total: 541 ms
Wall time: 506 ms

#reading the file using dask
import dask.dataframe as dd
%time df = dd.read_csv("balckfriday_train.csv")

CPU times: user 32.3 ms, sys: 3.63 ms, total: 35.9 ms
Wall time: 18 ms
```

The Black Friday dataset used here has 5,50,068 rows. **On using Dask, the read time reduced more than ten times as compared to using pandas!**

2. Finding value count for a particular column

```
df.Gender.Value_counts().compute()

M    414259
F    135809
Name: Gender, dtype: int64
```

3. Using groupby on the Dask dataframe

```
#finding maximum value of purchase for both genders

df.groupby(df.Gender).Purchase.max().compute()
```

```

Gender
F      23959
M      23961
Name: Purchase, dtype: int64

```

5.3 Dask ML

Dask ML provides scalable machine learning algorithms in python which are compatible with scikit-learn. Let us first understand how scikit-learn handles the computations and then we will look at how Dask performs these operations differently.



A user can perform parallel computing using scikit-learn (on a single machine) by setting the parameter `njobs = -1`. Scikit-learn uses *Joblib* to perform these parallel computations. ***Joblib* is a library in python that provides support for parallelization.** When you call the `.fit()` function, based on the tasks to be performed (whether it is a hyperparameter search or fitting a model), *Joblib* distributes the task over the available cores. To understand *Joblib* in detail, you can have a look at [this](#) documentation.

Even though parallel computations can be performed using scikit-learn, it cannot be scaled to multiple machines. On the other hand, Dask works well on a single machine and can also be scaled up to a cluster of machines.



Dask has a central task scheduler and a set of workers. The scheduler assigns tasks to the workers. Each worker is assigned a number of cores on which it can perform computations. The workers provide two functions:

- compute tasks as assigned by the scheduler
- serve results to other workers on demand

Below is an example that explains how a conversation between a scheduler and workers looks like (this has been given by one of the developers of Dask, Matthew Rocklin):

The central task scheduler sends jobs (python functions) to lots of worker processes, either on the same machine or on a cluster:

- *Worker A, please compute $x = f(1)$, Worker B please compute $y = g(2)$*
- *Worker A, when $g(2)$ is done please get y from Worker B and compute $z = h(x, y)$*

This should give you a clear idea about how Dask works. Now we will discuss about machine learning models and Dask-search CV!

5.3.1 ML models

Dask-ML provides scalable machine learning in python which we will discuss in this section. Implementation for the same will be covered in section 6. Let us first get our systems ready. Below are the installation steps for Dask-ML.

```
# Install with conda
conda install -c conda-forge dask-ml

# Install with pip
pip install dask-ml
```

1. Parallelize Scikit-Learn Directly

As we have seen previously, sklearn provides parallel computing (on a single CPU) using *Joblib*. In order to parallelize multiple sklearn estimators, you can directly use Dask by adding a few lines of code (without having to make modifications in the existing code).

The first step is to import *client* from *dask.distributed*. This command will create a local scheduler and worker on your machine.

```
from dask.distributed import Client
client = Client() # start a local Dask client
```

To read more about the Dask client, you can refer to [this document](#).

The next step will be to instantiate dask joblib in the backend. You need to import *parallel_backend* from *sklearn.joblib* like I have shown below.

```
import dask_ml.joblib
from sklearn.externals.joblib import parallel_backend
with parallel_backend('dask'):
    # Your normal scikit-learn code here
    from sklearn.ensemble import RandomForestClassifier
    model = RandomForestClassifier()
```

2. Reimplement Algorithms with Dask Array

For simple machine learning algorithms which use Numpy arrays, Dask ML re-implements these algorithms. Dask replaces numpy arrays with Dask arrays to achieve scalable algorithms. This has been implemented for:

- Linear models (linear regression, logistic regression, poisson regression)
- Pre-processing (scalers , transforms)
- Clustering (k-means, spectral clustering)

A. Linear model example

```
from dask_ml.linear_model import LogisticRegression

model = LogisticRegression()
model.fit(data, labels)
```

B. Pre-processing example

```
from dask_ml.preprocessing import OneHotEncoder

encoder = OneHotEncoder(sparse=True)
result = encoder.fit(data)
```

C. Clustering example

```
from dask_ml.cluster import KMeans
model = KMeans()
model.fit(data)
```

5.3.2 Dask-Search CV

Hyperparameter tuning is an important step in model building and can greatly affect the performance of your model. Machine learning models have multiple hyperparameters and it is not easy to figure out which parameter would work best for a particular case. Performing this task manually is generally a tedious process. In order to simplify the process, sklearn provides Gridsearch for hyperparameter tuning. The user is required to give the values for parameters and Gridsearch gives you the best combination of these parameters.

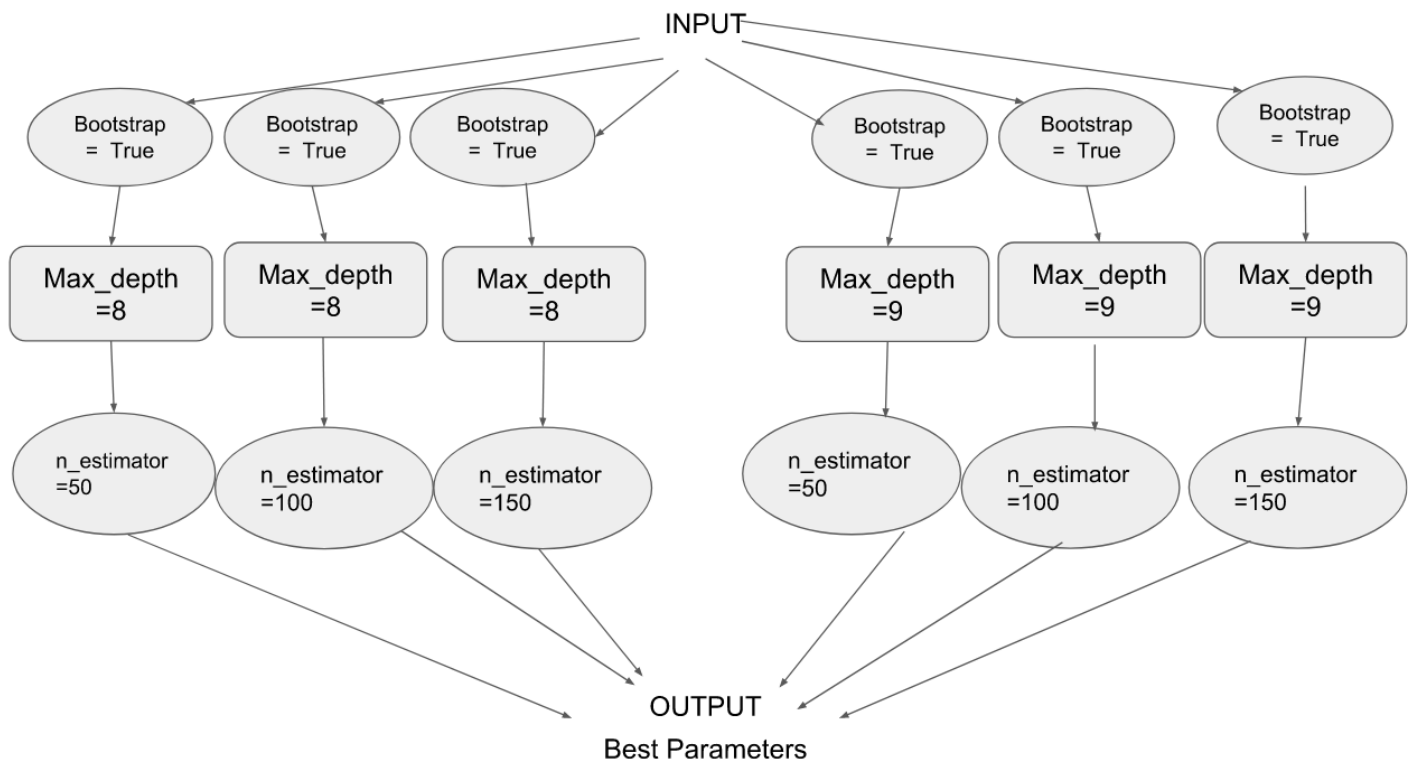
Consider an example where you choose a random forest technique to fit the dataset. Your model has three important tunable parameters – parameter 1, parameter 2 and parameter 3. You set the values for these parameters as:

Parameter 1 – Bootstrap = True

Parameter 2 – max_depth – [8, 9]

Parameter 3 – n_estimators : [50, 100 , 200]

sklearn Gridsearch : For each combination of the parameters, sklearn Gridsearch executes the tasks, sometimes ending up repeating a single task multiple times. As you can see from the below graph, this is not exactly the most efficient method:

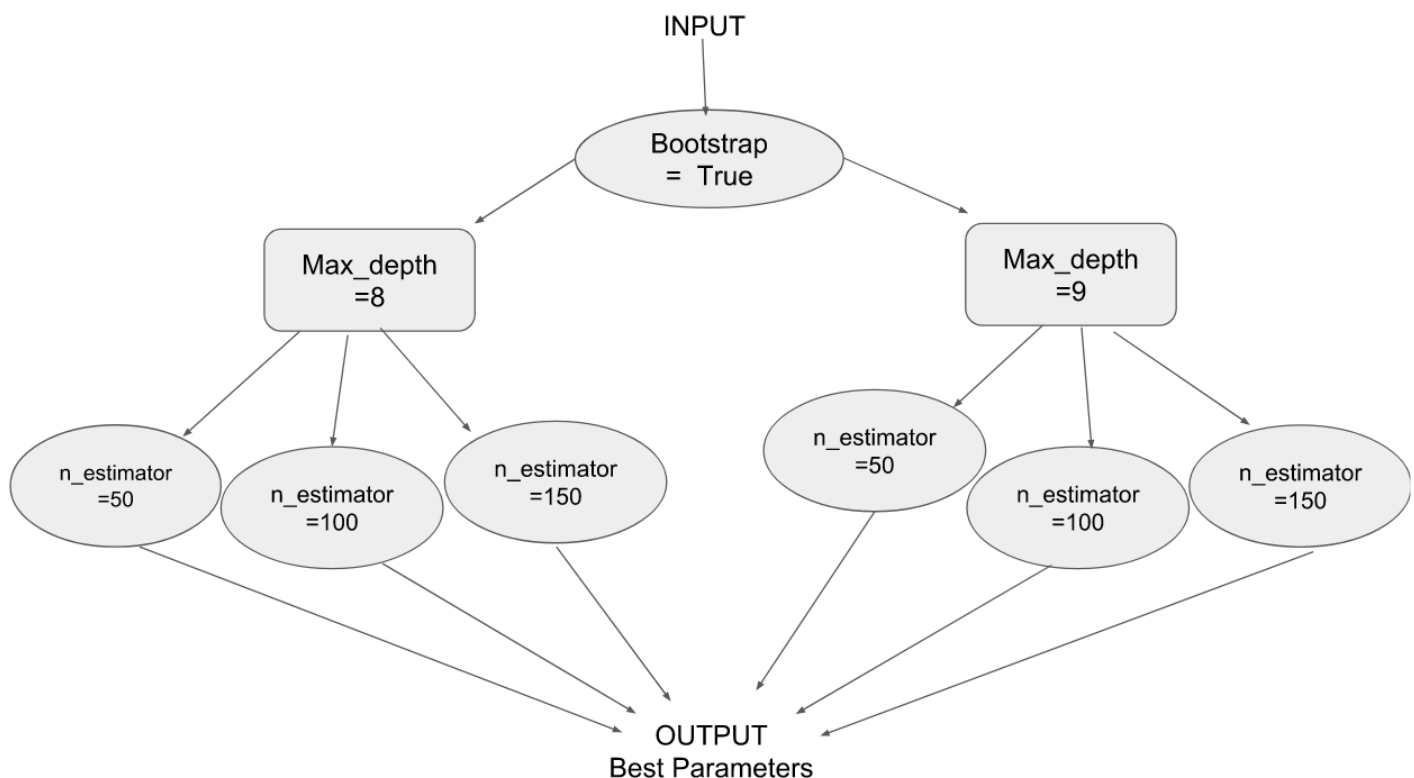


Dask-Search CV: Parallel to Gridsearch CV in sklearn, Dask provides a library called Dask-search CV (Dask-search CV is now included in Dask ML). It merges steps so that there are less repetitions. Below are the installation steps for Dask-search.

```
# Install with conda
conda install dask-searchcv -c conda-forge

# Install with pip
pip install dask-searchcv
```

The following graph explains the working of Dask-Search CV:



6. Solving a machine learning problem

We will implement what we have learned so far on the Black Friday dataset and see how it works. Data exploration and treatment is out of the scope of this article as I will only illustrate how to use Dask for a ML problem. In case you are interested in these steps, you can check out the below mentioned articles:

- [A Comprehensive Guide to Data Exploration](#)
- [Practical Guide on Data Preprocessing in Python](#)

1. Using a simple logistic regression model and making predictions

```
#reading the csv files
import dask.dataframe as dd
df = dd.read_csv('blackfriday_train.csv')
test=dd.read_csv("blackfriday_test.csv")

#having a look at the head of the dataset
df.head()

#finding the null values in the dataset
df.isnull().sum().compute()

#defining the data and target
categorical_variables = df[['Gender', 'Age', 'Occupation', 'City_Category', 'Stay_In_Current_City_Years', 'Marital_Status']]
target = df['Purchase']

#creating dummies for the categorical variables
data = dd.get_dummies(categorical_variables.categorize()).compute()

#converting dataframe to array
datanew=data.values

#fit the model
from dask_ml.linear_model import LinearRegression
lr = LinearRegression()
lr.fit(datanew, target)

#preparing the test data
test_categorical = test[['Gender', 'Age', 'Occupation', 'City_Category', 'Stay_In_Current_City_Years', 'Marital_Status']]
```

```
test_dummy = dd.get_dummies(test_categorical.categorize()).compute()
testnew = test_dummy.values

#predict on test and upload
pred=lr.predict(testnew)
```

This will give you the predictions on the given test set.

2. Using grid search and random forest algorithm to find the best set of parameters.

```
from dask.distributed import Client
client = Client() # start a local Dask client

import dask_ml.joblib
from sklearn.externals.joblib import parallel_backend
with parallel_backend('dask'):

    # Create the parameter grid based on the results of random search
    param_grid = {
        'bootstrap': [True],
        'max_depth': [8, 9],
        'max_features': [2, 3],
        'min_samples_leaf': [4, 5],
        'min_samples_split': [8, 10],
        'n_estimators': [100, 200]
    }

    # Create a based model
    from sklearn.ensemble import RandomForestRegressor
    rf = RandomForestRegressor()

# Instantiate the grid search model
import dask_searchcv as dc
grid_search = dc.GridSearchCV(estimator = rf, param_grid = param_grid, cv = 3)
grid_search.fit(data, target)
grid_search.best_params_
```

On printing `grid_search.best_params_` you will get the best combination of parameters for the given mode. I have varied only a few parameters here but once you are comfortable with using dask-search, I would suggest experimenting with more parameters while using multiple varying values for each parameter.

```
{'bootstrap': True,
 'max_depth': 8,
 'max_features': 2,
 'min_samples_leaf': 5,
 'min_samples_split': 8,
 'n_estimators': 200}
```

7. Spark vs Dask

One very common question that I have seen while exploring Dask is: How is Dask different from Spark and which one is preferred? There is no hard and fast rule that says one should use Dask (or Spark), but you can make your choice based on the features offered by them and whichever one suits your requirements more.

Here are some important differences between Dask and Spark :

Spark	Dask
Spark is written in Scala (programming language).	Dask is written in Python.
Provides support for R and python.	Only supports python.
Spark has its own ecosystem.	Dask is a component of python ecosystem.
Spark has its own APIs	Dask reuses the pandas' APIs
Easier to understand and implement for users familiar with Scala or SQL	Generally preferred by python practitioners.
Spark does not include support for multi-dimensional arrays natively (although some support for two-dimensional matrices may be found in MLlib.)	Dask fully supports the NumPy model for scalable multi-dimensional array.

Conclusion

I have recently started exploring the capabilities of Dask Python, and it's proving to be an amazing addition to my toolkit. It's comforting to know that, when dealing with large datasets, I don't have to navigate an entirely new tool. What sets [Dask Python](#) apart is its seamless integration with the familiar interface of Pandas. The best part is that the transition is remarkably smooth, with only a very slight (sometimes negligible) difference in the code. This feature makes Dask Python an excellent choice for scaling up my data workflows without the need for a steep learning curve.

There are innumerable tasks that one can perform using Dask thanks to the drastic reduction in processing time. Go ahead and explore this library and share your experience in the comments section below.

Frequently Asked Questions

Q1. Is Dask Faster than Pandas?

For smaller datasets that fit into memory, Pandas tends to be faster as it operates in-memory. However, as your dataset grows, Dask can outperform Pandas by distributing computations across multiple cores or machines, making it more scalable for handling large datasets. The choice depends on your data size and computational needs.

Q2.Which is Better Dask or Pyspark?

Choose Dask for scalable computations on a single machine or smaller clusters with easy Python integration. Opt for PySpark if you need robust distributed computing capabilities for handling large-scale datasets across clusters.



[Aishwarya Singh](#)

An avid reader and blogger who loves exploring the endless world of data science and artificial intelligence. Fascinated by the limitless applications of ML and AI; eager to learn and discover the depths of data science.

Big Data

Classification

Data Science

Intermediate

Libraries

Machine Learning

Pandas

Programming

Python

Python

Regression

Structured Data

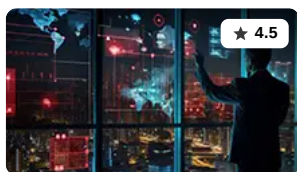
Supervised

Free Courses



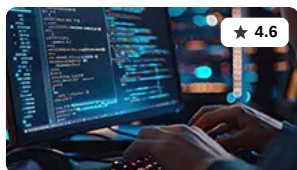
Generative AI - A Way of Life

Explore Generative AI for beginners: create text and images, use top AI tools, learn practical skills, and ethics.



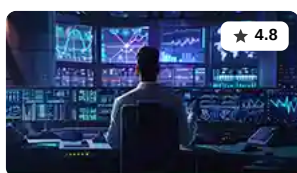
Getting Started with Large Language Models

Master Large Language Models (LLMs) with this course, offering clear guidance in NLP and model training made simple.



Building LLM Applications using Prompt Engineering

This free course guides you on building LLM apps, mastering prompt engineering, and developing chatbots with enterprise data.



Improving Real World RAG Systems: Key Challenges & Practical Solutions

Explore practical solutions, advanced retrieval strategies, and agentic RAG systems to improve context, relevance, and accuracy in AI-driven applications.



Microsoft Excel: Formulas & Functions

Master MS Excel for data analysis with key formulas, functions, and LookUp tools in this comprehensive course.

RECOMMENDED ARTICLES

[8 Must Know Spark Optimization Tips for Data En...](#)

[Top 100 Data Science Interview Questions and An...](#)

[Understanding Dask in Depth](#)

[99% of Python Developers Don't Know These Pan...](#)

[Comprehensive Introduction to Apache Spark, RDD...](#)

[PySpark for Beginners – Take your First S...](#)

[Hands-On Tutorial to Analyze Data using Spark SQL](#)

[Beginners Guide on Apache Spark & RDDs](#)

[Modin: Expedite Your Pandas Code with Single Ch...](#)

[Integration of Python with Hadoop and Spark](#)

Responses From Readers

What are your thoughts?...

Submit reply



Viraj

Thanks for sharing. It sounds like a promising library.

1

Show 1 reply



Nitin

Hello Aishwarya, That's a really awesome utility. Thanks for sharing it. I would like to make an edit in Section 6.2 below
***** # Instantiate the grid search model grid_search =
dcv.GridSearchCV(estimator = rf, param_grid = param_grid, cv = 3) ***** Here
we need to "import dask_searchcv as dcv" to make this command work. And before that one has to install in the env if it's not
available. Please update it for the benefit of others.

1

Show 1 reply

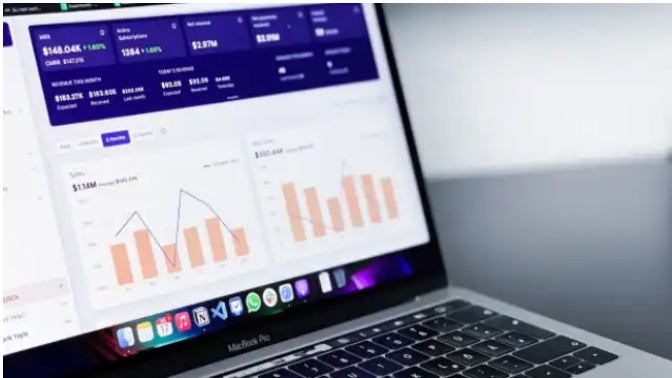


Jenarthanan

Write for us →

Write, captivate, and earn accolades and rewards for your work

- Reach a Global Audience
- Get Expert Feedback
- Build Your Brand & Audience
- Cash In on Your Knowledge
- Join a Thriving Community
- Level Up Your Data Science Game



Company

- About Us
- Contact Us
- Careers

Learn

- Free Courses
- AI&ML Program

Discover

- Blogs
- Expert Sessions
- Learning Paths
- Comprehensive Guides

Engage

- Community
- Hackathons

Pinnacle Plus Program
Agentic AI Program

Events
Podcasts

Contribute

Enterprise

Become an Author
Become a Speaker
Become a Mentor
Become an Instructor

Our Offerings
Trainings
Data Culture
AI Newsletter

